

Application Description – Checkers Game Project

This application represents a fully functional implementation of the traditional Checkers board game developed in C++ and supported by the NetID GUI framework. The goal of the project was to design a system that combines structured game logic with an interactive graphical interface, allowing users to play checkers against an AI opponent in a visually intuitive environment. The software follows an object-oriented architecture where each gameplay component is encapsulated into dedicated classes, ensuring modularity, maintainability, and efficiency.

```
1 gui::Window* createInitialWindow() override
2 {
3     return new MainWindow();
4 }
```

```
1 td::String trLang = appProperties->getValue("translation", "EN");
2 app.init(trLang);
3 return app.run();
```

Game Board and Graphical Rendering

The graphical representation of the game is handled through the NetID GUI canvas system. The application dynamically renders an 8×8 checkers board, including square positioning, scaling, and responsive resizing. The board automatically adjusts to window size changes, recalculating cell dimensions and maintaining correct piece alignment. Piece sprites, highlights, and overlays are drawn using image resources, ensuring a visually clear representation of game states such as selected pieces and available moves.

```
1 void updateModelSize(const gui::Size& sz) {
2     const float W = (float)sz.width;
3     const float H = (float)sz.height;
4     const float usableW = W * (1.0f - 2.0f * MARGIN);
5     const float usableH = H * (1.0f - 2.0f * MARGIN);
6     const float side = (usableW < usableH) ? usableW : usableH;
7     const float left = (W - side) * 0.5f;
8     const float bottom = (H - side) * 0.5f;
9
10    _boardPlacement.left = left;
11    _boardPlacement.bottom = bottom;
12    _boardPlacement.right = left + side;
13    _boardPlacement.top = bottom + side;
14
15    const float cell = side / (float)N;
16    _cellSize.width = (td::INT2)cell;
17    _cellSize.height = (td::INT2)cell;
18    _origin.x = (td::INT2)_boardPlacement.left;
19    _origin.y = (td::INT2)_boardPlacement.bottom;
20 }
21
```

Piece Movement Mechanics

Piece movement is governed by strict rule validation implemented in the core game engine. When a player selects a piece, the engine calculates all legal moves based on the current board state. Standard diagonal movement is enforced, and illegal moves are prevented. The system highlights valid destinations, guiding the player visually while maintaining rule correctness internally. Movement execution is animated smoothly across the board to improve user experience.

Capture System and Multi-Jump Logic

Capturing is a mandatory rule in checkers, and the application fully enforces it. The engine detects all capture opportunities using recursive search logic. When multiple captures are available, the player must perform them according to official rules. Multi-jump sequences are supported, allowing pieces to capture several opponents in a single turn. Captured pieces are removed dynamically during animation, ensuring synchronization between logic and visuals.



```
1  if (std::abs((int)b.r - (int)a.r) == 2 && std::abs((int)b.c - (int)a.c) == 2) {
2      int mr = ((int)a.r + (int)b.r) / 2;
3      int mc = ((int)a.c + (int)b.c) / 2;
4      _st.b[mr][mc] = 0;
5  }
```

King Promotion

Promotion mechanics are automatically triggered when a standard piece reaches the opposite end of the board. The engine detects promotion conditions at the end of a move sequence and upgrades the piece to a king. King pieces gain extended movement abilities, allowing both forward and backward diagonal movement. This upgrade is reflected visually through different piece sprites and logically through expanded move generation.



```
1  Piece finalPiece = _anim.movingPiece;
2      if (finalPiece == 1 && b.r == 0) finalPiece = 2;
3      if (finalPiece == -1 && b.r == N - 1) finalPiece = -2;
4      _st.b[b.r][b.c] = finalPiece;
5
```

Artificial Intelligence Opponent

The application includes a computer opponent powered by the Minimax algorithm enhanced with Alpha-Beta pruning. This AI evaluates possible game states to determine optimal moves. A heuristic evaluation function scores board positions based on piece count, king value, and positional advantage. Alpha-Beta pruning improves performance by eliminating unnecessary branches in the decision tree, allowing deeper search within reasonable computation time. AI processing runs on a separate thread, ensuring the graphical interface remains responsive.

Turn Management System

Turn handling is fully automated by the engine. After each completed move, the system switches active players. The interface updates accordingly, enabling or disabling user input depending on whose turn it is. If the AI is next, the system triggers AI computation automatically. This ensures smooth gameplay flow without requiring manual turn control.

Game End Detection

The engine continuously monitors win, loss, and draw conditions. A player loses when no legal moves remain or all pieces are captured. Draw detection is implemented using the “no capture move limit” rule, where a game ends if a predefined number of moves occur without any captures. Once an end condition is detected, gameplay stops immediately, and the final result is stored within the engine state.

Endgame Dialog System

When the match concludes, the application presents a dialog window informing the player whether they have won, lost, or drawn. This dialog offers options to start a new game or close the session. The system ensures the dialog appears only once per game by tracking result flags, preventing duplicate pop-ups during animation cycles.

Animation and Sound Effects

To enhance realism, the application integrates animation and audio feedback. Piece movements are interpolated smoothly between board cells using time-based animation. Sound effects play when moves occur and when the game concludes. These audiovisual cues improve immersion and provide immediate feedback for player actions.

User Interface Controls

The interface includes a menu bar and toolbar offering quick access to core actions such as starting or stopping a game, resetting the board, and opening settings. Icons and labels dynamically update based on game state, ensuring intuitive control. The settings dialog allows language switching and toolbar customization.

Localization and Language Support

The application supports multiple interface languages through an XML-based translation system. Users can change the application language from the settings dialog. Because translations are loaded during application initialization, a restart prompt appears after language changes to apply updates correctly.

Application Architecture

The system architecture separates responsibilities into layers. The engine handles logic, rules, AI, and state transitions. The view layer manages rendering, input handling, and animation. The window and dialog components coordinate user interaction, menus, and settings. This modular design allows independent testing and easier future expansion.